

Vanishing & Exploding Gradients, Worked Out by Hand

Why depth multiplies gradients, and how to read the per-layer norms

A complete numerical walk-through of *why* gradients in a deep network either fade to nothing or blow up to infinity — and how a handful of printed numbers tells you which is happening. Backprop multiplies one Jacobian factor per layer, so the gradient reaching an early layer scales like a single factor raised to the depth. We make that product concrete, then read it off the diagnostics you actually print: the per-layer gradient norm, the layer-to-layer ratio, and the global norm. Every number below was produced by the NumPy reference program in Appendix A.

What this document does. It fixes a typical per-layer Jacobian magnitude r and propagates it backward through a stack of layers by hand. With $r = 0.6$ the gradient *vanishes*; with $r = 1.5$ it *explodes*; with $r \approx 1$ it stays healthy. We then build two small per-layer norm tables — a vanishing net and a healthy net — and compute the exact diagnostics a `monitor_gradients` routine would print after `loss.backward()`.

How to read this. Each idea gets its own worked step with exact numbers, plus a coloured box isolating the intuition, the failure mode, or the fix. Read this to understand what the per-layer gradient norms in Module 10's gradient monitor *mean*, so that a stalled or NaN training run becomes a number you can diagnose rather than a mystery.

Concepts covered, in order: the chain rule as a product of Jacobians · the r^L depth-scaling law · the worked cases $r = 0.6, 1.0, 1.5$ · activations: sigmoid's 0.25 ceiling vs ReLU's 1 · per-layer gradient norm $\|g_\ell\|$ · the layer-to-layer ratio $\|g_\ell\|/\|g_{\ell+1}\|$ · the global norm $\sqrt{\sum_\ell \|g_\ell\|^2}$ · vanishing vs healthy norm tables · the fixes (init, ReLU, normalization, residuals, clipping).

1 The setup: backprop is a product over depth

Train a network of L layers. Forward, the data flows $x \rightarrow h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_L \rightarrow \text{loss}$. Backward, the chain rule sends the gradient the *other* way, and at each layer it is multiplied by that layer's local Jacobian $J_\ell = \partial h_\ell / \partial h_{\ell-1}$:

$$\frac{\partial \text{loss}}{\partial h_0} = \underbrace{J_1^\top J_2^\top \dots J_L^\top}_{L \text{ factors}} \frac{\partial \text{loss}}{\partial h_L}.$$

The gradient that finally reaches the *earliest* layer is a product of L matrices. Magnitudes multiply. If each factor scales a vector by roughly the same amount r (its typical singular value, weight-scale times activation-slope), then after L layers the gradient has been scaled by about r^L :

$$\|g_{\text{early}}\| \approx r^L \|g_{\text{late}}\|$$

Exponentials are unforgiving. r a hair below 1 collapses to zero over depth; r a hair above 1 races to infinity. That single inequality — is $r < 1$, $r = 1$, or $r > 1$? — is the whole subject.

Intuition

Gradients *multiply* as they flow back through depth; they do not add. Ten layers each shrinking the gradient by 0.6 do not subtract 6, they multiply to 0.6^{10} . Healthy training is the art of keeping that running product near 1: good initialisation sets each factor's scale to ≈ 1 , normalisation (BatchNorm/LayerNorm) re-centres it every layer, and residual connections add an identity path so the product cannot drift far from 1.

2 Step 1 — The depth-scaling law r^L , worked three ways

Fix the depth at $L = 10$ and read off r^L for three representative factors. The arithmetic is just repeated multiplication, shown cumulatively so you can watch the gradient drift layer by layer.

Vanishing, $r = 0.6$. Each layer keeps 60% of the gradient:

$$0.6^1=0.6, \quad 0.6^2=0.36, \quad 0.6^5=0.07776, \quad 0.6^{10} = \boxed{0.006}.$$

After ten layers only 0.6% of the gradient survives — the early layers feel almost nothing.

Healthy, $r = 1.0$. Each layer preserves the gradient exactly:

$$1.0^{10} = \boxed{1.000}.$$

The gradient that reaches layer 1 is the same size as the one leaving layer 10. Every layer can learn.

Exploding, $r = 1.5$. Each layer amplifies by 50%:

$$1.5^1=1.5, \quad 1.5^2=2.25, \quad 1.5^5=7.594, \quad 1.5^{10} = \boxed{57.665}.$$

The gradient is nearly $58\times$ larger at the early layers; a few more layers and it overflows to inf .

Factor r	r^{10}	Regime	What you see in training
0.6	0.006	vanishing	loss stalls; early layers frozen
1.0	1.000	healthy	all layers update; loss falls smoothly
1.5	57.665	exploding	loss spikes then goes NaN/inf

Mini-example — this operation in isolation

The cliff is steep because the exponent is the depth. Hold $r = 0.6$ and grow L : $0.6^5 = 0.07776$, $0.6^{10} = 0.006$, and $0.6^{20} \approx 3.66 \times 10^{-5}$. Doubling the depth squares the survival fraction. This is why “it worked at 6 layers but dies at 20” is the canonical deep-net complaint — nothing changed per layer; the exponent did.

3 Step 2 — Where r comes from: the activation slope

The factor r is roughly (weight scale) $\times$ (activation derivative). The activation’s *maximum slope* caps how large r can be, so the choice of nonlinearity is half the story.

Sigmoid has a hard ceiling of 0.25. For $\sigma(x) = 1/(1 + e^{-x})$ the derivative is $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, maximised where $\sigma(x) = 0.5$ (i.e. $x = 0$):

$$\sigma'(0) = 0.5 \times (1 - 0.5) = \boxed{0.25}.$$

That is the *best* case. Even ignoring the weights, a stack of sigmoids multiplies in at most 0.25 per layer, so the gradient cannot beat 0.25^L :

$$0.25^5 = 9.766 \times 10^{-4}, \quad 0.25^{10} = 9.537 \times 10^{-7}, \quad 0.25^{20} = 9.095 \times 10^{-13}.$$

A 10-deep sigmoid stack throttles the gradient by a factor of a million in the best case — and far worse once any unit saturates (where $\sigma' \rightarrow 0$).

ReLU’s slope is 1 on the active side. For $\text{ReLU}(x) = \max(0, x)$ the derivative is exactly 1 when $x > 0$ (and 0 when $x < 0$). On the active path it neither shrinks nor grows the gradient:

$$1^{10} = 1.000.$$

ReLU *preserves scale* through depth instead of strangling it — which is precisely why deep nets abandoned sigmoid hidden layers for ReLU/GELU.

Activation	max derivative	$(\max)^{10}$	Effect through depth
sigmoid	0.25	9.537×10^{-7}	strong vanishing, even at best
tanh	1.00 (at 0)	1.000	better, but saturates at the tails
ReLU	1.00 ($x > 0$)	1.000	preserves scale on the active path

Watch out

The sigmoid 0.25 is the *ceiling*, not the typical value. Real units sit away from $x = 0$, so the average slope is smaller — the course’s gradient monitor uses ≈ 0.18 for a representative sigmoid layer, giving $0.18^{10} \approx 3.57 \times 10^{-8}$: even more vanishing than the optimistic 0.25^{10} . Sigmoid (and tanh) in deep *hidden* layers is the classic “loss stalls, early layers never move” bug.

4 Step 3 — The three diagnostics you actually print

You never see r directly. After `loss.backward()` you see one gradient tensor per layer, and you summarise each by its **norm** $\|g_\ell\| = \sqrt{\sum_i g_{\ell,i}^2}$. Three derived quantities turn those norms into a diagnosis.

(1) **Per-layer gradient norm** $\|g_\ell\|$. Print it for every layer. A *healthy* net shows comparable norms across all layers; a sick net shows a monotone trend — tiny at the early layers (vanishing) or huge at the early layers (exploding).

(2) **Layer-to-layer ratio** $\|g_\ell\|/\|g_{\ell+1}\|$. This is exactly the empirical r between adjacent layers. A ratio near 1 everywhere is the goal; a ratio consistently < 1 marks vanishing, consistently > 1 marks exploding.

(3) **Global norm** $\sqrt{\sum_\ell \|g_\ell\|^2}$. One scalar for the whole gradient. It is the quantity gradient clipping rescales, and the first thing that prints as `nan/inf` when things explode.

When this is the right choice

Reach for these the moment training misbehaves. **Loss stalls / refuses to drop** $\Rightarrow$ suspect vanishing: print $\|g_\ell\|$ and look for early layers near zero. **Loss spikes or becomes NaN** $\Rightarrow$ suspect exploding: watch the global norm blow up. **Early layers “frozen”** (their weights barely change epoch to epoch) $\Rightarrow$ the layer-to-layer ratio is $\ll 1$ there. These three numbers localise the failure to a specific band of layers.

5 Step 4 — Two norm tables: vanishing vs healthy

Take a 6-layer stack. Layer 6 sits next to the loss, layer 1 is the earliest. We fix the late-layer norm at $\|g_6\| = 1.000$ and propagate backward by the empirical factor r , so $\|g_\ell\| = r^{6-\ell}$. This is the same arithmetic as Step 1, now displayed as the table you would actually print.

Vanishing net ($r = 0.6$). **Healthy net** ($r = 0.97$). Side by side:

Layer ℓ	position	$\ g_\ell\ $ (vanish, $r=0.6$)	$\ g_\ell\ $ (healthy, $r=0.97$)
6	nearest loss	1.000	1.000
5		0.600	0.970
4		0.360	0.941
3		0.216	0.913
2		0.130	0.885
1	earliest	0.078	0.859

Read the columns. In the vanishing net the earliest layer gets $\|g_1\| = 0.078$ while the last gets 1.000 — a *first/last ratio* of 0.078, so layer 1 receives roughly $13\times$ less gradient than layer 6 and barely learns. In the healthy net $\|g_1\| = 0.859$ against 1.000: a ratio of 0.859, close to the ideal 1.0, so every layer trains at a comparable rate.

The layer-to-layer ratios confirm the factor. Dividing each norm by the one below it returns

r exactly:

$$\text{vanish: } \frac{\|g_\ell\|}{\|g_{\ell+1}\|} = 0.600 \text{ (every layer),} \quad \text{healthy: } \frac{\|g_\ell\|}{\|g_{\ell+1}\|} = 0.970 \text{ (every layer).}$$

Global norms. Collapsing each table to one scalar:

$$\sqrt{\sum_\ell \|g_\ell\|^2} \Big|_{\text{vanish}} = 1.249, \quad \sqrt{\sum_\ell \|g_\ell\|^2} \Big|_{\text{healthy}} = 2.276.$$

The healthy net’s global norm is larger because more layers contribute real gradient; the vanishing net’s total is dominated by the last layer alone.

Intuition

The healthy column is nearly *flat* — 0.859, 0.885, ..., 1.000 — and flatness is the signal you want. When per-layer norms are all within a small factor of each other, gradient is reaching every layer and the whole network can learn. A column that decays by an order of magnitude top to bottom is the fingerprint of vanishing; one that grows by an order of magnitude is exploding.

6 Step 5 — The exploding case, and the clip that tames it

Flip the factor to $r = 1.5$ and the same 6-layer table inverts: the earliest layer now carries the *largest* norm.

Layer ℓ	position	$\ g_\ell\ $ (explode, $r=1.5$)
6	nearest loss	1.000
5		1.500
4		2.250
3		3.375
2		5.062
1	earliest	7.594

The global norm here is $\sqrt{\sum_\ell \|g_\ell\|^2} = 10.149$. Extend the same factor to $L = 10$ layers and the early-layer scale reaches $1.5^{10} = 57.665$ — well into the range where a single step overshoots and the loss prints `nan`.

Gradient clipping rescales the whole gradient. Clipping by global norm computes the total norm and, if it exceeds a threshold `max_norm`, scales *every* gradient by the same factor $c = \text{max_norm} / \|g\|_{\text{global}}$. With $\|g\|_{\text{global}} = 57.665$ and `max_norm` = 1.0:

$$c = \frac{1.0}{57.665} = \boxed{0.01734}.$$

Every per-layer gradient is multiplied by 0.01734, capping the global norm at 1.0 while *preserving the direction*. The explosive step becomes a safe one.

Watch out

Match the fix to the symptom. **Exploding** (global norm huge, loss NaN/inf): clip the gradient norm (e.g. `max_norm=1.0`), lower the learning rate, and fix the initialisation — the course's `Uniform(-5,5)` init explodes immediately. **Vanishing** (early-layer norms near zero, loss stalls): switch sigmoid/tanh hidden layers to ReLU/GELU, add BatchNorm/LayerNorm, add residual (skip) connections, and use Kaiming/Xavier init so each factor starts at $r \approx 1$. Clipping does *not* cure vanishing — it only bounds the top end.

7 Reading the diagnostics, at a glance

Signal	Vanishing	Healthy	Exploding
factor r	$r < 1$ (0.6)	$r \approx 1$ (0.97)	$r > 1$ (1.5)
r^{10}	0.006	1.000	57.665
early-layer $\ g_1\ $	0.078 (tiny)	0.859	7.594 (large)
ratio $\ g_\ell\ /\ g_{\ell+1}\ $	0.600	0.970	1.500
global norm (6 layers)	1.249	2.276	10.149
loss behaviour	stalls	falls smoothly	spikes / NaN
first fix	ReLU + init + norm	—	clip + lower lr

8 Equation cheat-sheet

Backprop as a product	$\frac{\partial \text{loss}}{\partial h_0} = J_1^\top \cdots J_L^\top \frac{\partial \text{loss}}{\partial h_L}$
Depth-scaling law	$\ g_{\text{early}}\ \approx r^L \ g_{\text{late}}\ $
Regime test	$r < 1$ vanish · $r = 1$ healthy · $r > 1$ explode
Sigmoid slope (max)	$\sigma'(x) = \sigma(1 - \sigma) \leq 0.25$
ReLU slope	$\text{ReLU}'(x) = 1$ for $x > 0$, else 0
Per-layer norm	$\ g_\ell\ = \sqrt{\sum_i g_{\ell,i}^2}$
Layer-to-layer ratio	$\ g_\ell\ /\ g_{\ell+1}\ \approx r$
Global norm	$\sqrt{\sum_\ell \ g_\ell\ ^2}$
Clip scale (by global norm)	$c = \text{max_norm}/\ g\ _{\text{global}}$

Appendix A Reference implementation

Every number above is reproducible from the following program. The first block is pure arithmetic (the r^L law and the norm tables); the second is the PyTorch hook pattern from Module 10 that prints per-layer gradient norms after a real `backward()`.

```
import numpy as np

# --- Step 1: the depth-scaling law  $r^L$  ---
L = 10
for r in (0.6, 1.0, 1.5):
    print(f"r={r}   r^{L} = {r**L:.3f}")      # 0.006, 1.000, 57.665

# --- Step 2: activation slopes ---
print(0.25**10)   # 9.537e-07   sigmoid best case vanishes
print(1.0**10)    # 1.000       ReLU preserves scale

# --- Step 4: per-layer norm tables ( $g_6 = 1.0$ ,  $g_1 = r^{(6-1)}$ ) ---
def table(r, nL=6):
    return {l: r**(nL - l) for l in range(nL, 0, -1)}

van = table(0.6)   # {6:1.0, 5:0.6, 4:0.36, 3:0.216, 2:0.130, 1:0.078}
heal = table(0.97) # {6:1.0, ..., 1:0.859}
for name, d in (("vanish", van), ("healthy", heal)):
    norms = [d[l] for l in range(6, 0, -1)]
    ratios = [d[l] / d[l + 1] for l in range(1, 6)]      # == r
    gnorm = np.sqrt(sum(v**2 for v in d.values()))        # global norm
    print(name, [round(x, 3) for x in norms], "global", round(gnorm, 3))
# vanish: global 1.249   healthy: global 2.276

# --- Step 5: clip an exploding global norm to max_norm = 1.0 ---
g_global = 1.5**10      # 57.665
c = 1.0 / g_global      # 0.01734   scale applied to every grad
print(round(g_global, 3), f"{c:.5f}")
```

PyTorch: print per-layer gradient norms with hooks. The course's `monitor_gradients` pattern (run after `loss.backward()`), plus the global-norm clip that tames the exploding case.

```
import torch, torch.nn as nn

def monitor_gradients(model):
    # After loss.backward(): print the norm of every parameter's gradient.
    for name, p in model.named_parameters():
        if p.grad is not None:
            n = p.grad.norm().item()
            tag = ""
            if n < 1e-7: tag = " <- VANISHING"
            if n > 100: tag = " <- EXPLODING"
            print(f" {name:24s} norm={n:.3e}{tag}")

# Optional: register tensor hooks to watch activations' gradients flow back.
acts = {}
def make_hook(name):
    def hook(grad):
        # grad = d loss / d (this activation)
        acts[name] = grad.norm().item()
    return hook
```

```
model = nn.Sequential(*[nn.Linear(32, 32), nn.ReLU()] * 6, nn.Linear(32, 1))
x = torch.randn(16, 32)
loss = model(x).sum()
loss.backward()
monitor_gradients(model)          # healthy: norms comparable across layers

# Tame an exploding run: clip the GLOBAL norm, preserving direction.
total = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
print(f"pre-clip global norm = {total:.3f}")    # scale applied = 1.0 / total
```

— end of document —